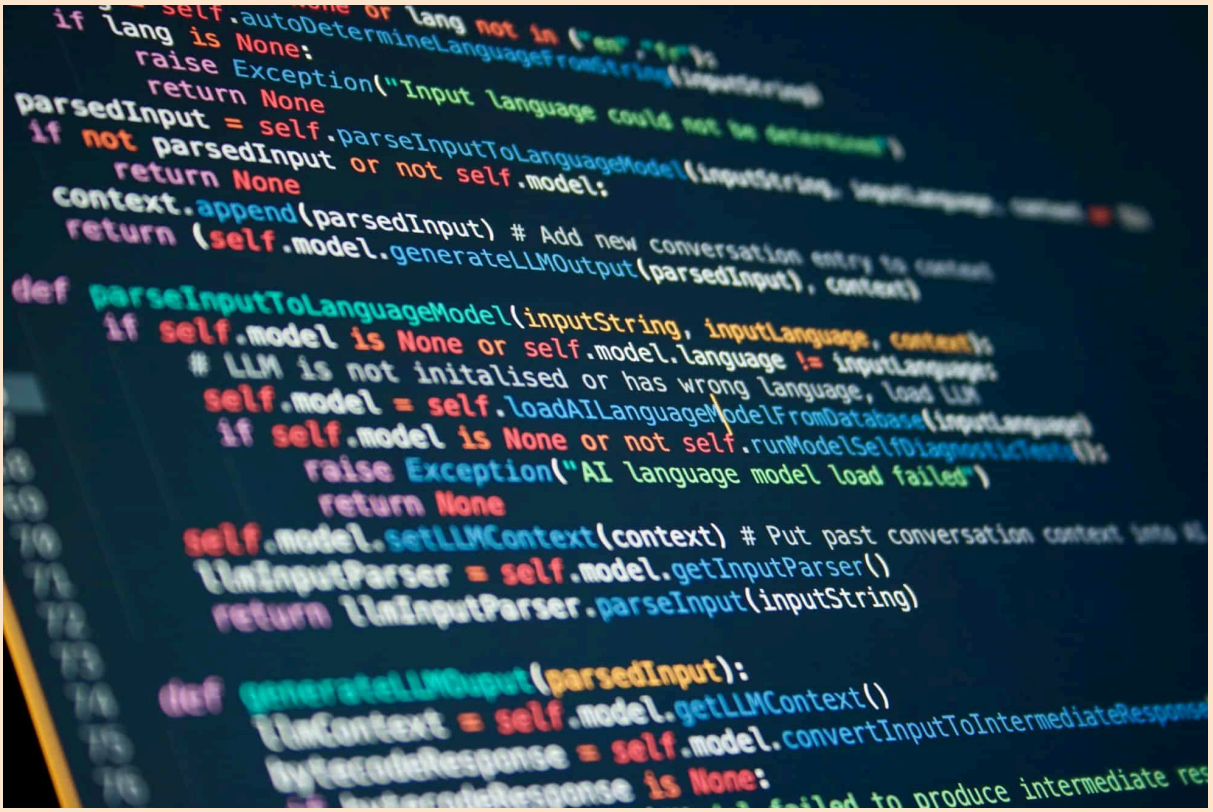
“Universidad Nacional Autónoma de México”
“Facultad de Ciencias”

# Compiladores.



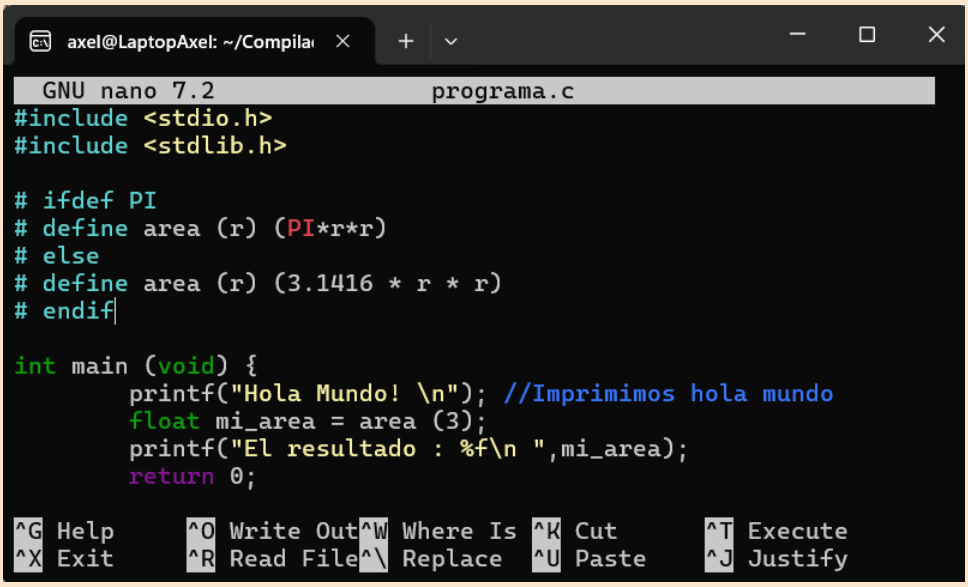

## Práctica 01.


## Irving Axel Cabrera Solís - 316031536

## Desarrollo.

- Deberá tener instalado el compilador gcc y trabajar en un ambiente Linux.
- Escriba el siguiente programa en lenguaje C (sin copiar y pegar) y nómbrelo programa.c

Procedemos a abrir la terminal de Linux Ubuntu y creamos con nano un nuevo programa llamado programa.c y copiamos el código de la práctica.




Use el siguiente comando: **cpp programa.c programa.i**

Revise el contenido de **programa.i** y conteste lo siguiente:

- ¿Qué ocurre cuando se invoca el comando cpp con esos argumentos?

  cuando ejecutamos este comando estamos activando el preprocesador de C en programa.c, y nos va a generar un archivo de salida llamado programa.i

  El preprocesador elimina los comentarios y expande los macros (define). Además hace las inclusiones de bibliotecas en el inicio como el #include stdio.h

- ¿Qué similitudes encuentra entre los archivos programa.c y programa.i?

```
# 0 "programa.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "programa.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h"
# 1 "/usr/include/features.h" 1 3 4
# 394 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 395 "/usr/include/features.h" 2 3 4
# 502 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 576 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 577 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 578 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 503 "/usr/include/features.h" 2 3 4
# 526 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
```

El archivo de programa.i es una versión muy extendida y muy minuciosa de programa.c donde se detalla cada inclusión de biblioteca por ejemplo

- ¿Qué pasa con los macros y los comentarios del código fuente original en programa.i?

Los comentarios fueron eliminados por completos y nuestros macros se expandieron minuciosamente al ser reemplazado por las instrucciones correspondientes

- Compare el contenido de programa.i con el de stdio.h e indique de forma general las similitudes entre ambos archivos.

Ambos tienen las mismas bibliotecas usadas en include, sólo que en programa.i está detallado en tu totalidad.

- ¿A qué etapa corresponde este proceso?

Es la primera parte del proceso de un programa en c, es decir es su preprocesamiento. El que sigue es la compilación.

**Ejecute la siguiente instrucción: gcc -Wall -S programa.i**

```
axel@LaptopAxel:~/Compiladores/P1$ gcc -Wall -S programa.i
programa.c: In function ‘main’:
programa.c:12:26: error: ‘r’ undeclared (first use in this funct
ion)
   12 |         float mi_area = area (3);
      |                          ^
programa.c:12:26: note: each undeclared identifier is reported o
nly once for each function it appears in
```

parece que el código copiado le falta alguna declaración así que incluí otra variable a propósito para que nos salte un warning con la instrucicón

```
axel@LaptopAxel:~/Compiladores/P1$ gcc -Wall -S programa.i
programa.c: In function ‘main’:
programa.c:13:15: warning: unused variable ‘numero’ [-Wunused-varia
ble]
   13 |         float numero= 20*20;
      |               ^~~~~~
```

- ¿Para qué sirve la opción -Wall?

Esta instrucción (Warning All) activa todas las advertencias en el compilador. Y ayuda a detectar problemas aunque no sean errores sintácticos.

- ¿Qué le indica a gcc la opción -S?

La opción -S le indica al compilador que debe detenerse después de la etapa de compilación, justo después de generar el código en lenguaje ensamblador, y no continuar con el ensamblado ni el enlazado.

- ¿Qué contiene el archivo de salida y cuál es su extensión?

```
        .file   "programa.c"
        .text
        .section        .rodata
.LC0:
        .string "Hola Mundo! "
.LC3:
        .string "El resultado : %f\n "
        .text
        .globl  main
        .type   main, @function
main:
.LFB6:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        subq    $16, %rsp
        leaq    .LC0(%rip), %rax
        movq    %rax, %rdi
        call    puts@PLT
        movss   .LC1(%rip), %xmm0
        movss   %xmm0, -8(%rbp)
        movss   .LC2(%rip), %xmm0
        movss   %xmm0, -4(%rbp)
        pxor    %xmm1, %xmm1
        cvtss2sd        -8(%rbp), %xmm1
        movq    %xmm1, %rax
        movq    %rax, %xmm0
        leaq    .LC3(%rip), %rax
        movq    %rax, %rdi
        movl    $1, %eax
        call    printf@PLT
        movl    $0, %eax
        leave
```

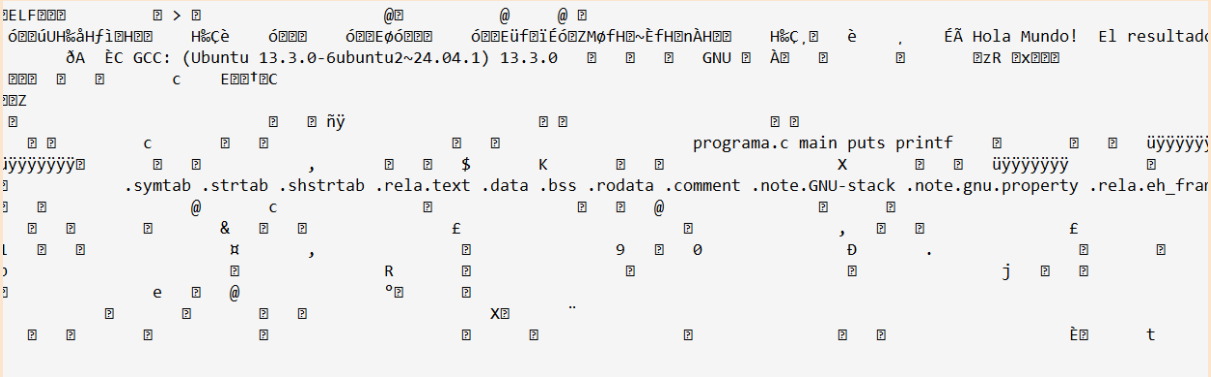
Nos genera un archivo de salida con extensión .s y si lo inspeccionamos veremos que contiene código en lenguaje ensamblador

- ¿A qué etapa corresponde este comando?

Corresponde a la etapa de compilación.

**Ejecute la siguiente instrucción: as programa.s -o programa.o**

- Antes de revisarlo, indique cuál es su hipótesis sobre lo que debe contener el archivo con extensión .o.
  Imagino que será un código directamente en lenguaje máquina (binario).
- Diga de forma general qué contiene el archivo programa.o y por qué se visualiza de esa manera.




Contiene instrucciones en binario, datos y metadatos del código.

Después de investigar vi que no se puede abrir con un editor de texto porque lo interpreta como basura ya que no es texto para ser leído por humanos sino código binario para la máquina.

- ¿Qué programa se invoca con as?
  El programa as es el ensamblador GNU (GNU Assembler). Forma parte del conjunto de herramientas Binutils.
- ¿A qué etapa corresponde la llamada a este programa?
  Estamos en la etapa de Ensamblado.

**Encuentre la ruta de los siguientes archivos en el equipo de trabajo:**

ld-linux-x86-64.so.2
Scrt1.o (o bien, crt1.o)
crti.o
crtbeginS.o
crtendS.o
crtn.o

**Ejecute el siguiente comando, sustituyendo las rutas que encontró en el paso anterior:**

**ld -o ejecutable -dynamic-linker /lib/ld-linux-x86-64.so.2 /usr/lib/Scrt1.o /usr/lib/crti.o programa.o -lc /usr/lib/crtn.o**

En caso de que el comando ld mande errores, investigue como enlazar un programa utilizando el comando ld. Y proponga una posible solución para llevar a cabo este proceso con éxito.
Describa el resultado obtenido al ejecutar el comando anterior.

```
axel@LaptopAxel:~/Compiladores/P1$ ld -o ejecutable -dynamic-linker /lib/l
d-linux-x86-64.so.2 /usr/lib/Scrt1.o /usr/lib/crti.o programa.o -lc /usr/l
ib/crtn.o
ld: cannot find /usr/lib/Scrt1.o: No such file or directory
ld: cannot find /usr/lib/crti.o: No such file or directory
```

Fue un proceso un poco complicado y no encontré una solución al problema.

Después de hacer prueba y error al parecer se logró el enlace manual y esto nos genera un nuevo archivo ejecutable.

**Quite el comentario de la macro #define PI en el código fuente original y conteste lo siguiente:**

- Genere nuevamente el archivo.i. De preferencia asigne un nuevo nombre.
- ¿Cambia en algo la ejecución final?

```
#define PI 3.1416
#include <iostream>
int main() {
    double radio = 5.0;
    double area = PI * radio * radio;
    std::cout << area << std::endl;
    return 0;
```

Si va a cambiar porque ahora la macro que activamos para reemplazar a PI en todas sus menciones anteriores.

**Escribe un segundo programa en lenguaje C en el que agregue 4 directivas del preprocesador de C (cpp)1. Las directivas elegidas deben jugar algún papel en el significado del programa, ser distintas entre sí y diferentes de las utilizadas en el primer programa (aunque no están prohibidas si las requieren).**

- Explique su utilidad general y su función en particular para su programa.

Se incluyó 4 directivas diferentes en el programa:
1 **#include <stdio.h>**
Esta nos sirve y es necesaria para el programa porque gestiona las entradas y las salidas
2 **#define SUMA(a, b) ((a) + (b))**
Como en el ejercicio de la práctica, al usarlo el compilador va a reemplazar a suma por la propia instrucción a+b
**3 #ifdef _DEBUG**
Es para algo llamado compilación condicional y nos va a servir para incluir o excluir códigos de bloque dependiendo de si un macro está definido o no lo está

**4 #ifndef PI**

Con este estamos asegurando que PI solo se defina si no ha sido definido previamente. Esto es útil si el programa es grande y varias partes pudieran definir PI.